

Documentación Externa

Generación de código MIPS

Estudiantes

Geovanni González Aguilar

Escuela de Ingeniería en Computación

Curso

IC5701 - Compiladores e Intérpretes

Profesor

Allan Rodríguez Dávila

Grupo

60

II Semestre 2026

Tabla de contenido

1. [Portada](#)
2. [Manual de usuario](#)
3. [Pruebas de funcionalidad](#)
4. [Descripción del problema](#)
5. [Librerías, módulos y manejo de archivos](#)
6. [Análisis de resultados](#)
7. [Algoritmos más importantes del proyecto](#)
8. [Bitácora de desarrollo basada en Git](#)

2. Manual de usuario

El proyecto es un compilador de línea de comandos que recibe un archivo fuente del lenguaje del curso (extensión `.chip`) y, además de los reportes de análisis, produce código de tres direcciones (código intermedio) y código ensamblador MIPS como código destino, ejecutable en QtSpim. Conforme al enunciado, la generación de MIPS se realiza a partir del código intermedio producido por el propio compilado.

2.1 Requisitos previos

Requisito	Detalle
Sistema operativo	Windows
JDK	Java Development Kit 17 o superior
Herramienta de construcción	Apache Maven 3.x.
Generador léxico	JFlex 1.9.1 (gestionado por Maven; no requiere instalación manual).
Generador sintáctico	Java CUP 11a/11b (incluido como <code>.jar</code> en <code>src/lib/</code>).
Framework de pruebas	JUnit Jupiter 5.10.2 (gestionado por Maven).
Simulador del código destino	QtSpim para ejecutar y verificar el archivo <code>.asm</code> generado.
Hardware	Sin requisitos especiales; cualquier equipo capaz de ejecutar el JDK 17.
Variables de entorno	<code>JAVA_HOME</code> apuntando al JDK 17 y <code>mvn/java</code> accesibles en el <code>PATH</code> .

2.2 Obtención del proyecto

```
git clone https://github.com/Geovanni-Gonzalez/Generacion-Codigo-Destino-MIPS.git
cd Generacion-Codigo-Destino-MIPS/programa
```

2.3 Instalación de dependencias

Las dependencias se resuelven automáticamente con Maven; no se requieren librerías adicionales a las declaradas:

```
mvn -q clean install
```

Este comando descarga `java-cup-runtime` y `junit-jupiter`, ejecuta JFlex y CUP para generar el analizador léxico y el sintáctico, compila el proyecto y corre la suite de pruebas.

2.4 Compilación

Desde la carpeta `programa/`:

```
mvn -q clean package
```

Artefactos generados:

Artefacto	Ubicación	Descripción
MiLexer.java	<code>target/generated-sources/jflex/</code>	Analizador léxico generado por JFlex a partir de <code>lexico.flex</code> .
Parser.java, sym.java	<code>target/generated-sources/cup/</code>	Analizador sintáctico y catálogo de tokens generados por CUP a partir de <code>sintactico.cup</code> .
proyecto-compiladores-1.0-SNAPSHOT.jar	<code>target/</code>	JAR ejecutable (empaquetado con <code>maven-shade-plugin</code> , clase principal <code>Main</code>).

Cómo identificar una compilación correcta: Maven finaliza con *BUILD SUCCESS* y el generador CUP reporta 38 conflicts detected. Los conflictos shift/reduce son esperados y se resuelven correctamente por desplazamiento; cualquier número distinto de 38 detiene la generación.

2.5 Ejecución

```
java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar <archivo_fuente> [directorio_salida]
```

Parámetro	Obligatorio	Descripción
<archivo_fuente>	Sí	Ruta al archivo .chip a compilar (permite elegir el archivo).
[directorio_salida]	No	Carpeta donde se escriben los reportes y el código generado. Por defecto: salida/.

Conforme al enunciado, el archivo destino MIPS conserva el nombre del fuente cambiando la extensión a .asm (por ejemplo, test.chip → test.asm).

Ejemplo completo:

```
java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar test/04_aritmeticas.chip salida
```

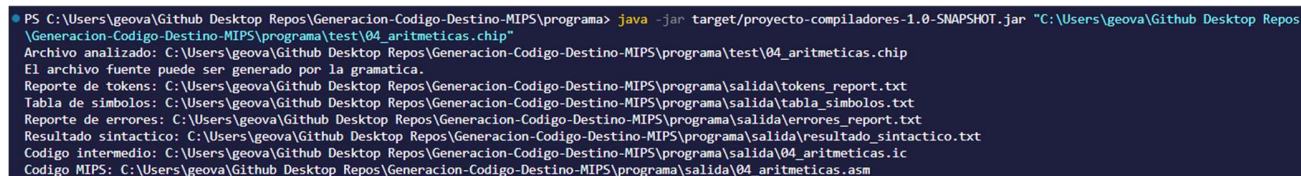
Archivos de salida generados (en el directorio indicado):

Archivo	Contenido
tokens_report.txt	Listado de tokens reconocidos (tabla, lexema, línea y columna).
tabla_simbolos.txt	Tabla de símbolos derivada del análisis.
errores_report.txt	Errores léxicos, sintácticos y semánticos, con conteo.
resultado_sintactico.txt	Veredicto de aceptación del programa por la gramática.
<base>.ic	Código intermedio (tres direcciones).
<base>.asm	Código destino MIPS (ejecutable en QtSpim).

Resultado esperado (consola):

```
Archivo analizado: ...\\test\\04_aritmeticas.chip
El archivo fuente puede ser generado por la gramatica.
Reporte de tokens: ...\\salida\\tokens_report.txt
...
Codigo intermedio: ...\\salida\\04_aritmeticas.ic
Codigo MIPS: ...\\salida\\04_aritmeticas.asm
```

Si el programa contiene errores, los archivos .ic y .asm no se generan y la consola indica El archivo fuente NO puede ser generado por la gramatica.



```
PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar "C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\04_aritmeticas.chip"
Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\04_aritmeticas.chip
El archivo fuente puede ser generado por la gramatica.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tokens_report.txt
Tabla de símbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\resultado_sintactico.txt
Codigo intermedio: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\04_aritmeticas.ic
Codigo MIPS: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\04_aritmeticas.asm
```

Figura 1. Ejecución del compilador desde línea de comandos y reporte de artefactos generados.

2.6 Uso del sistema

El flujo de uso es no interactivo (orientado a lotes):

1. **Preparar el archivo fuente.** Escribir un programa .chip (ver Anexo A).

2. **Invocar el compilador** con el comando de la sección 2.5.
3. **Procesamiento interno (automático)**: análisis léxico, sintáctico, semántico, generación de código intermedio y generación de MIPS.
4. **Consultar resultados**: abrir los reportes *.txt*, *el <base>.ic* y *el <base>.asm*.
5. **Ejecutar el código destino** (opcional): *cargar <base>.asm* en **QtSpim** y ejecutarlo para observar el comportamiento del programa.
6. **Cierre**: el programa termina por sí mismo tras escribir los archivos.

2.7 Errores frecuentes

Error o situación	Posible causa	Solución
Uso: java -jar ... y termina	No se indicó el archivo fuente.	Proporcionar la ruta del .chip como primer argumento.
La ruta del archivo fuente no es valida	Ruta inexistente o mal escrita.	Verificar la ruta; usar comillas si contiene espacios.
El archivo fuente NO puede ser generado por la gramática	Errores léxicos, sintácticos o semánticos.	Revisar errores_report.txt y corregir el fuente.
.ic/.asm no se generan	El análisis no fue aceptado.	Corregir los errores reportados; el código solo se genera para programas válidos.
el programa debe contener exactamente un metodo main	Falta empty ~ __main__< >.	Declarar el __main__ obligatorio.
More conflicts encountered than expected (build)	Cambio en la gramática.	Ajustar -expect en pom.xml tras validar conflictos benignos.
No hay registros temporales MIPS disponibles	Expresión demasiado compleja para \$t0–\$t5.	Dividir la expresión en pasos más simples.

3. Pruebas de funcionalidad

Funcionalidades verificadas: análisis léxico, análisis sintáctico con Recuperación en Modo Pánico, análisis semántico, generación de código intermedio, generación de MIPS, y el módulo de clases.

Entorno de pruebas: Windows; JDK 17; Maven; suite JUnit 5 (CompiladorTest) ejecutada con mvn test; verificación de la corrida del MIPS en QtSpim.

Datos de prueba: programas .chip reales en programa/test/ (01_minimo_valido.chip ... 31_clase_constructor_args.chip), válidos e inválidos.

Criterio de éxito: una prueba se considera aprobada cuando el veredicto de aceptación, el código intermedio y/o el MIPS coinciden con lo esperado según las aserciones de *CompiladorTest*.

Prueba funcional 1: Compilación de programa aritmético válido

- **Objetivo:** verificar que un programa válido es aceptado y produce código intermedio y MIPS.
- **Funcionalidad evaluada:** flujo completo léxico→MIPS, precedencia de operadores.
- **Precondiciones:** proyecto compilado.
- **Datos de entrada:** test/04_aritmeticas.chip.
- **Procedimiento:** 1) compilar; 2) verificar ; 3) inspeccionar IR y MIPS.
- **Resultado esperado:** el IR contiene `_t0 = 5 * 2` y `_t1 = 10 + _t0` (multiplicación antes que suma); el MIPS contiene .data, .text, .globl main, main:, d_main_x: .word 0 y el epílogo li \$v0, 10.

```
programa > test > 04_aritmeticas.chip
1  empty ~ __main__<| |>
2  |:
3      int ~ x <- 10 + 5 * 2 !
4      int ~ y <- <|10 + 5|> * 2 !
5      int ~ z <- 2 ^ 3 % 3 !
6      float ~ f <- -2.5 + 4.0 / 2.0 !
7      ++x !
8      --y !
9  :|
10
```

Figura 2. Código fuente 04_aritmeticas.chip

```

programa > salida > 04_aritmeticas.ic
1  //Codigo intermedio
2  // Fecha: 2026-06-28
3  // Archivo fuente: 04_aritmeticas.chip
4  // Integrantes: Giovanni Gonzalez
5
6      begin_function __main__
7      declare int x
8      _t0 = 5 * 2
9      _t1 = 10 + _t0
10     x = _t1
11     declare int y
12     _t2 = 10 + 5
13     _t3 = _t2 * 2
14     y = _t3
15     declare int z
16     _t4 = 2 ^ 3
17     _t5 = _t4 % 3
18     z = _t5
19     declare float f
20     _t6 = -2.5
21     _t7 = 4.0 / 2.0
22     _t8 = _t6 + _t7
23     f = _t8
24     _t9 = load x
25     _t10 = _t9 + 1
26     x = _t10
27     _t11 = load y
28     _t12 = _t11 - 1
29     y = _t12
30     end_function __main__
31

```

Figura 2. Código intermedio generado en base del .chip

Código MIPS generado

```

.data
d_main_x: .word 0
d_main_y: .word 0
d_main_z: .word 0
d_main_f: .float 0.0
d_main_t0: .word 0
d_main_t1: .word 0
d_main_t2: .word 0
d_main_t3: .word 0
d_main_t4: .word 0
d_main_t5: .word 0
d_main_t6: .float 0.0
d_main_t7: .float 0.0
d_main_t8: .float 0.0
d_main_t9: .word 0
d_main_t10: .word 0
d_main_t11: .word 0

```

```

d_main_t12: .word 0
_flt_0: .float 2.5
_flt_1: .float 4.0
_flt_2: .float 2.0

.text
.globl main

main:
    li $t0, 5
    li $t1, 2
    mul $t2, $t0, $t1
    sw $t2, d_main_t0
    li $t0, 10
    lw $t1, d_main_t0
    add $t2, $t0, $t1
    sw $t2, d_main_t1
    move $t0, $t2
    sw $t0, d_main_x
    li $t0, 10
    li $t1, 5
    add $t2, $t0, $t1
    sw $t2, d_main_t2
    move $t0, $t2
    li $t1, 2
    mul $t2, $t0, $t1
    sw $t2, d_main_t3
    move $t0, $t2
    sw $t0, d_main_y
    li $t0, 2
    li $t1, 3
    li $t2, 1
_mips_pow_0:
    blez $t1, _mips_pow_fin_1
    mul $t2, $t2, $t0
    addiu $t1, $t1, -1
    j _mips_pow_0
_mips_pow_fin_1:
    sw $t2, d_main_t4
    move $t0, $t2
    li $t1, 3
    div $t0, $t1
    mfhi $t2
    sw $t2, d_main_t5
    move $t0, $t2

```

```
sw $t0, d_main_z
l.s $f0, _flt_0
neg.s $f2, $f0
s.s $f2, d_main_t6
l.s $f0, _flt_1
l.s $f2, _flt_2
div.s $f4, $f0, $f2
s.s $f4, d_main_t7
l.s $f0, d_main_t6
l.s $f2, d_main_t7
add.s $f4, $f0, $f2
s.s $f4, d_main_t8
mov.s $f0, $f4
s.s $f0, d_main_f
lw $t0, d_main_x
sw $t0, d_main_t9
li $t1, 1
add $t2, $t0, $t1
sw $t2, d_main_t10
move $t0, $t2
sw $t0, d_main_x
lw $t0, d_main_y
sw $t0, d_main_t11
li $t1, 1
sub $t2, $t0, $t1
sw $t2, d_main_t12
move $t0, $t2
sw $t0, d_main_y
main_fin:
li $v0, 10
syscall
```

Ejecución QtSpim

FP Regs		Int Regs [10]	
Int Regs [10]			
PC	=	4194700	
EPC	=	0	
Cause	=	0	
BadVAddr	=	0	
Status	=	805371664	
HI	=	2	
LO	=	2	
R0	[r0]	=	0
R1	[at]	=	268500992
R2	[v0]	=	10
R3	[v1]	=	0
R4	[a0]	=	3
R5	[a1]	=	2147479228
R6	[a2]	=	2147479244
R7	[a3]	=	0
R8	[t0]	=	29
R9	[t1]	=	1
R10	[t2]	=	29
R11	[t3]	=	0
R12	[t4]	=	0
R13	[t5]	=	0
R14	[t6]	=	0
R15	[t7]	=	0
R16	[s0]	=	0
R17	[s1]	=	0
R18	[s2]	=	0
R19	[s3]	=	0
R20	[s4]	=	0
R21	[s5]	=	0
R22	[s6]	=	0
R23	[s7]	=	0
R24	[t8]	=	0
R25	[t9]	=	0
R26	[k0]	=	0
R27	[k1]	=	0
R28	[gp]	=	0

Figura 3. Estado de los registros enteros en QtSpim después de la ejecución del código MIPS. Se observa la finalización correcta del programa, el resultado final de $y = 29$ y el residuo $z = 2$ almacenado en el registro HI

FP Regs		Int Regs [10]
FP Regs		
FIR	=	38912
FCSR	=	0
FCCR	=	0
FEXR	=	0
Single Precision		
FG0	=	-0.500000
FG1	=	0.000000
FG2	=	2.000000
FG3	=	0.000000
FG4	=	-0.500000
FG5	=	0.000000
FG6	=	0.000000
FG7	=	0.000000
FG8	=	0.000000
FG9	=	0.000000
FG10	=	0.000000
FG11	=	0.000000
FG12	=	0.000000
FG13	=	0.000000
FG14	=	0.000000
FG15	=	0.000000
FG16	=	0.000000
FG17	=	0.000000
FG18	=	0.000000
FG19	=	0.000000
FG20	=	0.000000
FG21	=	0.000000
FG22	=	0.000000
FG23	=	0.000000
FG24	=	0.000000
FG25	=	0.000000
FG26	=	0.000000
FG27	=	0.000000
FG28	=	0.000000
FG29	=	0.000000
FG30	=	0.000000

Figura 4. Estado de los registros de punto flotante en QtSpim después de la ejecución. Los registros \$f0 y \$f4 contienen el resultado final -0.500000, confirmando la evaluación correcta de la expresión $-2.5 + 4.0 / 2.0$

Prueba funcional 2: Detección de errores y Recuperación en Modo Pánico

- **Objetivo:** comprobar que los programas inválidos son rechazados, no generan código y se recuperan en modo pánico para reportar múltiples errores.
- **Funcionalidad evaluada:** reporte de errores y recuperación sintáctica.
- **Datos de entrada:** *test/13_error_lexico.chip*, *test/14_error_sintactico.chip*, *test/26_error_sin_cascada.chip*.
- **Procedimiento:** compilar; revisar reporte de errores.
- **Resultado esperado:** programas rechazados.

```
programa > test > 13_error_lexico.chip
1  empty ~ __main__ <| |>
2  |:
3      int ~ x <- 10 !
4      &
5      cout <|x|> !
6  :|
7
```

Figura 5. Código fuente archivo 13_error_lexico.chip

```
PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar "C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\13_error_lexico.chip"
Error lexico [línea 4, col 5]: símbolo no reconocido '&'
Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\13_error_lexico.chip
El archivo fuente NO puede ser generado por la gramática.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tokens_report.txt
Tabla de símbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\resultado_sintactico.txt
Codigo intermedio no generado por errores de analisis.
Codigo MIPS no generado por errores de analisis.
```

Figura 6. Ejecución del archivo, reporta los errores detectados(léxicos), genera los archivos de reportes y no genera los archivos .ic ni .asm

```
programa > test > 14_error_sintactico.chip
1  empty ~ __main__ <| |>
2  |:
3      int ~ x <- 10
4      x <- x + !
5      cout <|x|> !
6  :|
7
```

Figura 7. Código fuente archivo 14_error_sintactico.chip

```
PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar "C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\14_error_sintactico.chip"
Error sintactico [línea 4, col 5]: (#1) token 'x': se inserto '!' para recuperar el analisis
Error sintactico [línea 4, col 14]: (#2) token '!': Syntax error
Error sintactico [línea 5, col 12]: (#3) token 'x': Expresion invalida despues de '+'
Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\14_error_sintactico.chip
El archivo fuente NO puede ser generado por la gramática.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tokens_report.txt
Tabla de símbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\resultado_sintactico.txt
Codigo intermedio no generado por errores de analisis.
Codigo MIPS no generado por errores de analisis.
```

Figura 8. Ejecución del archivo, reporta los errores detectados(sintácticos), genera los archivos de reportes y no genera los archivos .ic ni .asm

```

programa > test > 26_error_sin_cascada.chip
1  empty ~ __main__<| |>
2  |:
3      int ~ a <- x + 1 !
4      int ~ b <- x - 1 !
5      int ~ c <- x * 2 !
6      int ~ d <- x / 2 !
7      int ~ e <- x % 2 !
8      if <|x|>
9          |:
10             int ~ f <- x + 3 !
11             int ~ g <- x - 3 !
12         :|
13         do
14             |:
15                 int ~ h <- x * 4 !
16             :|
17         while <|x|> !
18         int ~ i <- x + 5 !
19     :|
20

```

Figura 9. Código fuente archivo 26_error_sin_cascada.chip

```

PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar "C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\26_error_sin_cascada.chip"
Error semantico [línea 3, col 1]: variable 'x' no declarada. Declare la variable antes de usarla y verifique la escritura de su nombre
Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\26_error_sin_cascada.chip
El archivo fuente NO puede ser generado por la gramática.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tokens_report.txt
Tabla de símbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\resultado_sintactico.txt
Codigo intermedio no generado por errores de analisis.
Codigo MIPS no generado por errores de analisis.
PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa>

```

Figura 10. Ejecución del archivo, reporta los errores detectados(semánticos), genera los archivos de reportes y no genera los archivos .ic ni .asm

Prueba funcional 3: Reglas semánticas del lenguaje

- **Objetivo:** validar tipado fuerte, condiciones booleanas, retorno obligatorio y unicidad de main.
- **Datos de entrada:** 15_doble_main.chip, 17_main_tipo_invalido.chip, 21_asignaciones_tipado_fuerte.chip, 23_condiciones_no_bool.chip, 25_returns_tipado.chip.
- **Procedimiento:** compilar cada caso y verificar el rechazo con el mensaje semántico correspondiente.
- **Resultado esperado:** todos rechazados con el diagnóstico adecuado.

```

programa > test > 15_doble_main.chip
1  empty ~ __main__<| |>
2  |:
3  :|
4
5  empty ~ __main__<| |>
6  |:
7  :|
8

```

Figura 11. Código fuente de archivo 15_doble_main.chip

```

PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar "C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\15_doble_main.chip"
Error semantico [linea 5, col 1]: '_main_' ya esta declarado en este alcance
Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\15_doble_main.chip
El archivo fuente NO puede ser generado por la gramatica.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tokens_report.txt
Tabla de simbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\resultado_sintactico.txt
Codigo intermedio no generado por errores de analisis.
Codigo MIPS no generado por errores de analisis.

```

Figura 12. Ejecución del archivo, reporta los errores semánticos correspondientes y genera los archivos de reportes

```

programa > test > 17_main_tipo_invalido.chip
1  int ~ __main__ <| |>
2  |:
3      return ~ 0 !
4  :|
5

```

Figura 13. Código fuente de archivo 17_main_tipo_invalido.chip

```

PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar "C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\17_main_tipo_invalido.chip"
Error sintactico [linea 1, col 15]: (#1) token '<|': El procedimiento '__main__' debe ser de tipo 'empty'
Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\17_main_tipo_invalido.chip
El archivo fuente NO puede ser generado por la gramatica.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tokens_report.txt
Tabla de simbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\resultado_sintactico.txt
Codigo intermedio no generado por errores de analisis.
Codigo MIPS no generado por errores de analisis.

```

Figura 14. Ejecución del archivo, reporta los errores correspondientes y genera los archivos de reportes

```

programa > test > 21_asignaciones_tipado_fuerte.chip
1  empty ~ __main__ <| |> |:
2      int ~ x <- 1!
3      float ~ y <- 1.5!
4      x <- 2!
5      x <- 2.5!
6      y <- 3.5!
7      noDeclarada <- 1!
8  :|
9

```

Figura 15. Código fuente del archivo 21_asignaciones_tipado_fuerte.chip

```

PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar "C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\21_asignaciones_tipado_fuerte.chip"
Error semantico [linea 5, col 1]: asignacion incompatible para 'x': se esperaba tipo int y se obtuvo tipo float
Error semantico [linea 7, col 1]: variable 'noDeclarada' no declarada. Declare la variable antes de usarla y verifique la escritura de su nombre
Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\21_asignaciones_tipado_fuerte.chip
El archivo fuente NO puede ser generado por la gramatica.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tokens_report.txt
Tabla de simbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\resultado_sintactico.txt
Codigo intermedio no generado por errores de analisis.
Codigo MIPS no generado por errores de analisis.

```

Figura 16. Ejecución del archivo, reporta los errores correspondientes y genera los archivos de reportes

```
programa > test > 23_condiciones_no_bool.chip
```

```
1  empty ~ __main__<| |>
2  |:
3      int ~ x <- 1 !
4      if <|x|>
5      |:
6          y <- 2 !
7      :|
8      do
9      |:
10         x <- x + 1 !
11     :|
12     while <|"texto"|> !
13 :|
14
```

Figura 17. Código fuente del archivo 23_condiciones_no_bool.chip

```
PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar "C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\23_condiciones_no_bool.chip"
Error semantico [linea 6, col 1]: variable 'y' no declarada. Declare la variable antes de usarla y verifique la escritura de su nombre
Error semantico [linea 4, col 1]: la condicion debe ser de tipo bool, pero se encontro tipo int
Error semantico [linea 12, col 1]: la condicion debe ser de tipo bool, pero se encontro tipo string
Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\23_condiciones_no_bool.chip
El archivo fuente NO puede ser generado por la gramatica.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tokens_report.txt
Tabla de simbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\resultado_sintactico.txt
Codigo intermedio no generado por errores de analisis.
Codigo MIPS no generado por errores de analisis.
```

Figura 18. Ejecución del archivo, reporta los errores correspondientes y genera los archivos de reportes

```
programa > test > 25_returns_tipado.chip
```

```
1  int ~ retornaEnteroOk<| |>
2  |:
3      return 5 !
4  :|
5
6  int ~ retornaFloatEnInt<| |>
7  |:
8      return 5.0 !
9  :|
10
11 void ~ voidConValor<| |>
12 |:
13     return 5 !
14 :|
15
16 int ~ intSinValor<| |>
17 |:
18     return !
19 :|
20
21 empty ~ __main__<| |>
22 |:
23     cout <|retornaEnteroOk<| |>> !
24 :|
25
```

Figura 19. Código fuente del archivo 25_returns_tipado.chip

```

PS C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa> java -jar target/proyecto-compiladores-1.0-SNAPSHOT.jar "C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\25_returns_tipado.chip"
Error sintactico [linea 3, col 12]: (#1) token '5': Syntax error
Error sintactico [linea 6, col 5]: (#2) token '~': Sentencia invalida; se descarta hasta el siguiente '!'
Error sintactico [linea 8, col 12]: (#3) token '5.0': Syntax error
Error sintactico [linea 11, col 6]: (#4) token '~': Sentencia invalida; se descarta hasta el siguiente '!'
Error sintactico [linea 13, col 12]: (#5) token '5': Syntax error
Error sintactico [linea 16, col 5]: (#6) token '~': Sentencia invalida; se descarta hasta el siguiente '!'
Error semantico [linea 18, col 1]: la funcion de tipo int debe retornar un valor
Error semantico [linea 1, col 1]: la funcion de tipo int debe retornar un valor en todas las rutas de ejecucion
Error semantico [linea 6, col 1]: la funcion de tipo int debe retornar un valor en todas las rutas de ejecucion
Error semantico [linea 16, col 1]: la funcion de tipo int debe retornar un valor en todas las rutas de ejecucion
Archivo analizado: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\test\25_returns_tipado.chip
El archivo fuente NO puede ser generado por la gramatica.
Reporte de tokens: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tokens_report.txt
Tabla de simbolos: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\tabla_simbolos.txt
Reporte de errores: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\errores_report.txt
Resultado sintactico: C:\Users\geova\Github Desktop Repos\Generacion-Codigo-Destino-MIPS\programa\salida\resultado_sintactico.txt
Codigo intermedio no generado por errores de analisis.
Codigo MIPS no generado por errores de analisis.

```

Figura 20. Ejecución del archivo, reporta los errores correspondientes y genera los archivos de reportes

Prueba funcional 4: Clases — estructura y MIPS

- **Objetivo:** verificar definición de clase, new, acceso a campos y su traducción a MIPS con asignación en heap.
- **Datos de entrada:** test/28_clase_basica.chip.
- **Procedimiento:** compilar; verificar (new Punto, 12, field p @4:int = 10) y MIPS (li \$v0, 9, li \$a0, 12, acceso 4(\$t0)).
- **Resultado esperado:** objeto reservado en heap de 12 bytes, campos escritos por *offset*.

```

programa > test > 28_clase_basica.chip
1  class ~ Punto |:
2      int ~ x !
3      int ~ y !
4  :|
5
6  empty ~ __main__<| |>
7  |:
8      Punto ~ p <- new Punto<| |> !
9      p.x <- 10 !
10     p.y <- 20 !
11     cout <| p.x |> !
12     cout <| p.y |> !
13 :|
14

```

Figura 21. Código fuente de clase básica

```

programa > salida > 28_clase_basica.ic
1  //Codigo intermedio
2  // Fecha: 2026-06-28
3  // Archivo fuente: 28_clase_basica.chip
4  // Integrantes: Giovanni Gonzalez
5
6      begin_function __main__
7      declare objeto p
8      _t0 = new Punto, 12
9      p = _t0
10     field p @4:int = 10
11     field p @8:int = 20
12     _t1 = field p @4:int
13     print _t1
14     _t2 = field p @8:int
15     print _t2
16     end_function __main__
17

```

Figura 22. Código intermedio generado

Código MIPS generado

```

.data
d_main_p: .word 0
d_main_t0: .word 0
d_main_t1: .word 0
d_main_t2: .word 0

.text
.globl main

main:
    li $v0, 9
    li $a0, 12
    syscall
    sw $v0, d_main_t0
    move $t0, $v0
    sw $t0, d_main_p
    li $t1, 10
    sw $t1, 4($t0)
    lw $t0, d_main_p
    li $t1, 20
    sw $t1, 8($t0)
    lw $t0, d_main_p
    lw $t1, 4($t0)
    sw $t1, d_main_t1
    move $a0, $t1
    li $v0, 1

```

```

syscall
lw $t0, d_main_p
lw $t1, 8($t0)
sw $t1, d_main_t2
move $a0, $t1
li $v0, 1
syscall
main_fin:
li $v0, 10
syscall

```

Prueba funcional 5: Clases — uso (métodos, this, constructor, despacho)

- **Objetivo:** verificar métodos, paso de this, constructor que inicializa y llamada con valor de retorno.
- **Datos de entrada:** test/30_clase_metodos.chip.
- **Procedimiento:** compilar; verificar IR (begin_function Punto_Punto, parameter objeto this, call Punto_Punto, 3, call Punto_sumar, 1) y MIPS (jal_fn_Punto_Punto, jal_fn_Punto_sumar); ejecutar en QtSpim.
- **Resultado esperado:** el constructor inicializa x=10, y=20; sumar retorna 30; el programa imprime **30**.

```

programa > test > 30_clase_metodos.chip
1  class ~ Punto |:
2      int ~ x !
3      int ~ y !
4
5      empty ~ Punto<| int ~ a, int ~ b |>
6      |:
7          this.x <- a !
8          this.y <- b !
9      :|
10
11     int ~ sumar<| |>
12     |:
13         return ~ this.x + this.y !
14     :|
15 :|
16
17 empty ~ __main__<| |>
18 |:
19     Punto ~ p <- new Punto<| 10, 20 |> !
20     cout <| p.sumar<| |> |> !
21 :|
22

```

Figura 23. Código fuente archivo 30_clase_metodos.chip

```

programa > salida > 30_clase_metodos.ic
1  //Codigo intermedio
2  // Fecha: 2026-06-28
3  // Archivo fuente: 30_clase_metodos.chip
4  // Integrantes: Geovanni Gonzalez
5
6      begin_function Punto_Punto
7      parameter objeto this
8      parameter int a
9      parameter int b
10     _t0 = load a
11     field this @4:int = _t0
12     _t1 = load b
13     field this @8:int = _t1
14     end_function Punto_Punto
15     begin_function Punto_sumar
16     parameter objeto this
17     _t2 = field this @4:int
18     _t3 = field this @8:int
19     _t4 = _t2 + _t3
20     return _t4
21     end_function Punto_sumar
22     begin_function __main__
23     declare objeto p
24     _t5 = new Punto, 12
25     param _t5
26     param 10
27     param 20
28     call Punto_Punto, 3
29     p = _t5
30     param p
31     _t6 = call Punto_sumar, 1
32     print _t6
33     end_function __main__
34

```

Figura 24. Código intermedio generado

Código MIPS generado

```

.data
d_Punto_Punto_this: .word 0
d_Punto_Punto_a: .word 0
d_Punto_Punto_b: .word 0
d_Punto_sumar_this: .word 0
d_main_p: .word 0
d_Punto_Punto_t0: .word 0
d_Punto_Punto_t1: .word 0
d_Punto_sumar_t2: .word 0
d_Punto_sumar_t3: .word 0
d_Punto_sumar_t4: .word 0
d_main_t5: .word 0
d_main_t6: .word 0

.text

```

```

.globl main

_fn_Punto_Punto:
    addiu $sp, $sp, -4
    sw $ra, 0($sp)
    lw $t0, 12($sp)
    sw $t0, d_Punto_Punto_this
    lw $t0, 8($sp)
    sw $t0, d_Punto_Punto_a
    lw $t0, 4($sp)
    sw $t0, d_Punto_Punto_b
    lw $t0, d_Punto_Punto_a
    sw $t0, d_Punto_Punto_t0
    lw $t0, d_Punto_Punto_this
    lw $t1, d_Punto_Punto_t0
    sw $t1, 4($t0)
    lw $t0, d_Punto_Punto_b
    sw $t0, d_Punto_Punto_t1
    lw $t0, d_Punto_Punto_this
    lw $t1, d_Punto_Punto_t1
    sw $t1, 8($t0)

_fn_Punto_Punto_fin:
    lw $ra, 0($sp)
    addiu $sp, $sp, 4
    jr $ra

_fn_Punto_sumar:
    addiu $sp, $sp, -4
    sw $ra, 0($sp)
    lw $t0, 4($sp)
    sw $t0, d_Punto_sumar_this
    lw $t1, 4($t0)
    sw $t1, d_Punto_sumar_t2
    lw $t0, d_Punto_sumar_this
    lw $t1, 8($t0)
    sw $t1, d_Punto_sumar_t3
    lw $t0, d_Punto_sumar_t2
    lw $t1, d_Punto_sumar_t3
    add $t2, $t0, $t1
    sw $t2, d_Punto_sumar_t4
    move $v0, $t2
    j _fn_Punto_sumar_fin

_fn_Punto_sumar_fin:
    lw $ra, 0($sp)
    addiu $sp, $sp, 4

```

```
    jr $ra

main:
    li $v0, 9
    li $a0, 12
    syscall
    sw $v0, d_main_t5
    move $t0, $v0
    addiu $sp, $sp, -4
    sw $t0, 0($sp)
    li $t0, 10
    addiu $sp, $sp, -4
    sw $t0, 0($sp)
    li $t0, 20
    addiu $sp, $sp, -4
    sw $t0, 0($sp)
    jal _fn_Punto_Punto
    addiu $sp, $sp, 12
    lw $t0, d_main_t5
    sw $t0, d_main_p
    addiu $sp, $sp, -4
    sw $t0, 0($sp)
    jal _fn_Punto_sumar
    addiu $sp, $sp, 4
    sw $v0, d_main_t6
    move $a0, $v0
    li $v0, 1
    syscall
main_fin:
    li $v0, 10
    syscall
```

FP Regs		Int Regs [10]	
		Int Regs [10]	
PC	=	4194688	
EPC	=	0	
Cause	=	0	
BadVAddr	=	0	
Status	=	805371664	
HI	=	0	
LO	=	0	
R0	[r0]	=	0
R1	[at]	=	268500992
R2	[v0]	=	10
R3	[v1]	=	0
R4	[a0]	=	30
R5	[a1]	=	2147479220
R6	[a2]	=	2147479236
R7	[a3]	=	0
R8	[t0]	=	10
R9	[t1]	=	20
R10	[t2]	=	30
R11	[t3]	=	0
R12	[t4]	=	0
R13	[t5]	=	0
R14	[t6]	=	0
R15	[t7]	=	0
R16	[s0]	=	0
R17	[s1]	=	0
R18	[s2]	=	0
R19	[s3]	=	0
R20	[s4]	=	0
R21	[s5]	=	0
R22	[s6]	=	0
R23	[s7]	=	0
R24	[t8]	=	0
R25	[t9]	=	0
R26	[k0]	=	0
R27	[k1]	=	0
R28	[gp]	=	0
R29	[sp]	=	2147479216

Figura 25. Estado de los registros enteros después de ejecutar la creación y utilización de un objeto de la clase Punto. Los registros \$t0 = 10, \$t1 = 20 y \$t2 = 30 evidencian la lectura y suma correcta de los campos, mientras que \$a0 = 30 corresponde al valor mostrado en la consola de QtSpim.

Console

30

Figura 26. Salida en consola de QtSpim

4. Descripción del problema

4.1 Contexto

Un grupo de desarrolladores desea crear un nuevo lenguaje imperativo, ligero, orientado a operaciones básicas para la configuración de chips y el desarrollo de sistemas empujados, una industria en crecimiento que demanda lenguajes ligeros y potentes. El presente trabajo corresponde al Proyecto #3 del curso y aborda la fase de generación de código destino (MIPS) de dicho lenguaje, reutilizando el *front-end* (analizadores JFlex y CUP) construido en los Proyectos I y II, y partiendo del código de tres direcciones generado por el Proyecto II.

4.2 Problema

Se requiere un compilador que, a partir de un programa fuente del lenguaje (archivos .chip, compuestos por una secuencia de declaraciones de procedimientos y exactamente un método main), realice lo siguiente (requerimientos del enunciado):

- a) Preservar y corregir los alcances de los Proyectos I y II.
- b) Indicar si el archivo fuente puede o no ser generado por la gramática, considerando gramática, sintaxis y semántica.
- c) Reportar los errores léxicos, sintácticos y semánticos mediante Recuperación en Modo Pánico.
- d) Escribir en un archivo el código MIPS correspondiente, semánticamente equivalente al código fuente.

4.3 Solución propuesta

Se implementó un compilador por fases en Java que transforma un archivo .chip en código MIPS ejecutable en QtSpim:

- **Entrada:** un archivo fuente .chip.
- **Procesos:** análisis léxico (JFlex), análisis sintáctico LALR con Recuperación en Modo Pánico (CUP) entrelazado con análisis semántico, construcción de un árbol sintáctico abstracto (AST), generación de código de tres direcciones, y traducción de ese código a MIPS con asignación de memoria, registros, E/S.
- **Salidas:** reportes de tokens, tabla de símbolos y errores; el código intermedio (.ic) y el código destino (.asm).
- **Modalidad:** la generación de MIPS parte del código intermedio producido por el propio compilador
- **Ventajas:** arquitectura modular, reportes claros, suite de pruebas automatizada y un código intermedio legible.

4.4 Alcance

- Tipos: int, float, bool, char, string.
- Expresiones aritméticas, relacionales y lógicas, con precedencia correcta.
- Control de flujo: if/else, do-while, switch/case/default, break.
- Funciones con parámetros y retorno; procedimiento principal __main__ obligatorio.
- Arreglos bidimensionales.
- Entrada/salida: cin, cout.
- Clases: definición con campos; instanciación con new y asignación dinámica en heap; acceso a campos por offset; métodos, this, constructores que inicializan y despacho estático de métodos.
- Generación de código intermedio y de código destino MIPS ejecutable en QtSpim.

4.5 Limitaciones

- **Clases:** no se implementan herencia ni polimorfismo. El acceso a campos dentro de métodos es mediante this.campo explícito; los campos deben declararse antes que los métodos. No se admiten campos de tipo objeto (composición), arreglos de objetos, ni objetos como parámetro/retorno (salvo this). No hay liberación de memoria (free) ni verificación de puntero nulo.

4.6 Objetivos

Objetivo general: Desarrollar, en Java y con las herramientas JFlex y CUP, el generador de código destino MIPS del lenguaje .chip, a partir del código de tres direcciones del Proyecto II, de modo que el código generado sea semánticamente equivalente al fuente y ejecutable en QtSpim.

Objetivos específicos:

1. Preservar y corregir los alcances léxico, sintáctico y semántico de los Proyectos I y II.
2. Determinar si un archivo fuente es generado por la gramática, considerando sintaxis y semántica.
3. Reportar los errores léxicos, sintácticos y semánticos mediante Recuperación en Modo Pánico.
4. Generar el código intermedio de tres direcciones como base de la traducción.
5. Escribir en un archivo .asm (mismo nombre del fuente) el código MIPS equivalente, ejecutable en QtSpim.
6. Incorporar el manejo de clases (estructura y uso) a través de las cuatro fases y su traducción a MIPS
7. Validar el funcionamiento mediante una suite de pruebas automatizadas.

5. Librerías, módulos y manejo de archivos

Conforme al enunciado, se especifican todas las librerías necesarias para compilar y ejecutar. No se requieren librerías adicionales fuera de las herramientas oficiales (JFlex, CUP) y JUnit (solo para pruebas); el resto se cubre con la biblioteca estándar de Java. Los archivos JFlex (lexico.flex) y CUP (sintactico.cup) se incluyen en el proyecto, como exige el enunciado.

Librería	Versión	Propósito	Parte del proyecto donde se utiliza
JFlex	1.9.1	Generador de analizadores léxicos.	src/lexico/lexico.flex → MiLexer.
Java CUP (generador)	11a	Generador de analizadores sintácticos LALR.	src/sintactico/sintactico.cup → Parser, sym.
java-cup-runtime	11b-20160615	Soporte en ejecución del parser CUP .	Paquete sintactico y el lexer.
JUnit Jupiter	5.10.2	Framework de pruebas unitarias.	src/test/java/pipeline/CompiladorTest.java.
maven-shade-plugin	3.5.0	Empaqueta un JAR ejecutable con la clase Main.	Fase package.
build-helper-maven-plugin	3.4.0	Agrega las fuentes generadas al classpath.	Fase initialize.
maven-surefire-plugin	3.5.4	Ejecuta las pruebas.	Fase test.
maven-antrun-plugin	3.1.0	Invoca CUP para generar el parser.	Fase generate-sources.
Biblioteca estándar de Java	JDK 17	java.nio.file (E/S), java.util (colecciones, pila de alcances), java.util.regex (clasificación de operandos).	Todo el proyecto.

6. Análisis de resultados

6.1 Resultados obtenidos

Se implementó un compilador funcional de extremo a extremo que convierte programas .chip en código MIPS ejecutable en QtSpim. El funcionamiento se comprobó con una suite de 36 pruebas JUnit (0 fallos) que cubre programas válidos e inválidos, reglas semánticas, generación de IR y de MIPS, y el módulo de clases.

6.2 Objetivos alcanzados

Objetivo	Estado	Evidencia	Explicación
1. Preservar alcances I/II	Alcanzado	semantico/, pruebas	Tabla de símbolos con alcances apilados.
2. Aceptar/rechazar por gramática	Alcanzado	resultado_sintactico.txt	Veredicto de aceptación reportado.
3. Errores + Modo Pánico	Alcanzado	errores_report.txt; sintactico.cup	Sincronización por bloques e inserción de !.
4. Código intermedio	Alcanzado	GeneradorCodigoIntermedio; .ic	IR explícito con temporales y operaciones.
5. Código MIPS equivalente, QtSpim	Alcanzado	Paquete mips; .asm	Memoria, registros, E/S y syscalls.
6. Clases (estructura y uso) — bonus	Parcialmente alcanzado	28–31_*.chip	Campos, new, métodos, this, constructor y despacho estático; sin herencia ni polimorfismo.
7. Validación con pruebas	Alcanzado	CompiladorTest (36 pruebas)	Suite automatizada en verde.

6.3 Objetivos no alcanzados (o parcialmente alcanzados)

- Clases — parcialmente alcanzado. Quedaron pendientes la herencia y el polimorfismo
 - Por qué no se completó: Factor del tiempo en contra de la fecha y hora límite del proyecto.

7. Algoritmos más importantes del proyecto

7.1 Análisis sintáctico LALR con Recuperación en Modo Pánico

- Propósito: reconocer la estructura del programa, construir el AST y recuperarse de errores sin abortar (técnica exigida por el enunciado).
- Entradas: flujo de tokens. Salidas: ProgramaNodo (AST) y lista de errores.
- Estructuras de datos: pila LR (CUP), ArrayList de errores, tabla de símbolos.
- Descripción: analizador LALR generado por CUP; un envoltorio (siguienteTokenConRecuperacion) inserta ! cuando detecta el inicio de una nueva sentencia sin terminador; las producciones con error sincronizan hasta un punto seguro (:|, !).
- Pseudocódigo:

```
ALGORITMO AnalisisSintactico
  MIENTRAS haya tokens HACER
    token <- siguienteTokenConRecuperacion()
    SI token inicia nueva unidad Y el anterior puede cerrar sentencia ENTONCES
      insertar '!' y reportar recuperacion
    ejecutar accion LALR (shift/reduce)
    SI hay error ENTONCES
      descartar hasta simbolo de sincronizacion ; reportar error // Modo Panico
  FIN MIENTRAS
  RETORNAR AST
FIN ALGORITMO
```

- **Archivo:** programa/src/sintactico/sintactico.cup.

7.2 Análisis semántico con tabla de símbolos por alcances

- Propósito: verificar tipos, declaraciones, unicidad de main y rutas de retorno (req. a).
- Estructuras de datos: pila de tablas hash (Stack<HashMap<String, Simbolo>>).
- Descripción: se apila un alcance al entrar a programa/función/método/bloque y se desapila al salir; la búsqueda recorre la pila desde el tope; el tipado es recursivo (evaluarTipo).
- Pseudocódigo:

```
ALGORITMO BuscarSimbolo(nombre)
  PARA alcance DESDE el tope HASTA la base HACER
    SI alcance contiene nombre ENTONCES RETORNAR alcance[nombre]
  reportar "no declarado" ; RETORNAR simbolo ERROR
FIN ALGORITMO
```

- **Archivo:** programa/src/java/semantico/. **Complejidad:** $O(d)$, con d = profundidad de alcances.

7.3 Generación de código intermedio (tres direcciones)

- Propósito: convertir el AST en instrucciones simples (base de la traducción a MIPS).
- Descripción: cada expresión binaria genera un temporal y una operación; el control de flujo genera etiquetas y saltos; las clases generan métodos Clase_metodo con this y los objetos generan NEW/STORE_FIELD/LOAD_FIELD.
- Pseudocódigo:

```
ALGORITMO GenerarExpresionBinaria(nodo)
  izq <- Generar(nodo.izquierda) ; der <- Generar(nodo.derecha)
  t <- nuevoTemporal() ; emitir (operacion(nodo.operador), t, izq, der) ; RETORNAR t
FIN ALGORITMO
```

- **Archivo:** programa/src/java/intermedio/GeneradorCodigoIntermedio.java.

7.4 Generación de código MIPS

- Propósito: traducir el IR a ensamblador MIPS ejecutable en QtSpim .
- Descripción: (1) AnalizadorIRMIPS recolecta declaraciones, propaga tipos y asigna una dirección global a cada variable/temporal; (2) se emite .data; (3) cada instrucción se traduce según su tipo (transferencia, operación, control, E/S, funciones, objetos); (4) un paso peephole (OptimizadorMIPS) elimina redundancias (p. ej. sw/lw al mismo registro → move).
- Pseudocódigo:

```
ALGORITMO GenerarMIPS(IR)
  analisis <- AnalizadorIRMIPS.analizar(IR)
  emitir .data ; emitir .text, .globl main
  PARA CADA instruccion EN IR HACER traducir segun su operacion
  RETORNAR peephole(salida)
FIN ALGORITMO
```

- Archivos: programa/src/java/mips/ (GeneradorMIPS, AnalizadorIRMIPS, traductores, EmisorDatosMIPS, OptimizadorMIPS).

7.5 Modelo de objetos en MIPS

- Propósito: representar objetos con identidad propia y traducir su uso a MIPS.
- Estructuras de datos: *layout* por clase (offset y tipo de cada campo; tamaño total); palabra 0 reservada para futura vtable; campos desde *offset* 4.
- Descripción: new Clase → syscall 9 (sbrk) reserva el tamaño de la clase; el puntero (\$v0) se guarda en la variable. obj.campo carga el puntero y usa lw/sw reg, offset(\$ptr). Los métodos se compilan como funciones que reciben this como primer parámetro por pila; el constructor (Clase_Clase) inicializa los campos; el despacho es estático (jal _fn_Clase_metodo).
- Pseudocódigo:

```
ALGORITMO TraducirNew(clase, tamano, destino)
  li $v0, 9 ; li $a0, tamano ; syscall    // reservar en heap
  sw $v0, etiqueta(destino)             // guardar el puntero
FIN ALGORITMO
```

- Archivos: programa/src/java/mips/TraductorObjetosMIPS.java, intermedio/GeneradorCodigoIntermedio.java.

8. Bitácora de desarrollo basada en Git

Link: [Geovanni-Gonzalez/Generacion-Codigo-Destino-MIPS: Proyecto 3 del curso compiladores e intérpretes](#)